PROJECT REPORT

Personal Finance Advisor Bot

An AI-powered budgeting assistant built with Flask and Google Gemini

Program	SkillWallet - AI Specialist (BFSI)
Project	Personal Finance Advisor Bot
Technologies	Python, Flask, Google Gemini API, SQLite, HTML, CSS, JavaScript
Source code	https://github.com/alwaysonkar/Personal-Finance-Advisor-Bot
Submitted by	_Onkar Ramesh Kadam

Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Tools and Technologies
5. System Architecture
6. Project Structure
7. Working of the System
8. Database Design
9. Routes (API)
10. Input Validation and Error Handling
11. User Interface
12. Testing
13. Limitations and Future Scope
14. Conclusion

1. Introduction

Managing money is hard for many people. Most do not track where their income goes each month, and a financial advisor is expensive. The Personal Finance Advisor Bot is a small web application that gives budgeting advice in a few seconds. The user types in a monthly income, the amount spent in each category and a savings goal. The app sends these numbers to Google Gemini, a large language model, and shows a recommended budget, a check of every category against a common guideline, and a short list of specific actions in rupees.

The project was made as part of the SkillWallet AI Specialist (BFSI) program, where generative AI is applied to banking, financial services and insurance use cases.

2. Problem Statement

People who earn a regular income often overspend on rent, food, dining out or entertainment and save less than they would like. They need a simple tool that (a) compares their spending with sensible guidelines, (b) proposes a realistic budget for a chosen goal, and (c) keeps a record of past analyses so they can see progress month by month.

3. Objectives

- Collect monthly income, expenses and a savings goal through a clean web form.
- Use a generative AI model to produce a budget, a category-wise analysis and practical suggestions.
- Calculate the totals and savings rate in the application itself so the key numbers are always exact.
- Store every analysis in a database and show a monthly summary and recent history.
- Validate all input and handle AI or network failures without crashing.
- Keep the code organised in separate modules so it is easy to read, test and extend.

4. Tools and Technologies

Tool	Used for
Python 3	Main programming language of the backend
Flask 3.0.3	Web framework: routes, JSON responses, HTML templates
Google Gemini API (google-genai)	Generates the budget, analysis and suggestions. Model is set in .env, default gemini-2.5-flash
SQLite (sqlite3)	Stores past analyses in a local file, no server needed
python-dotenv	Loads the API key and settings from the .env file
HTML, CSS, JavaScript	Frontend form, result display and charts of saved data. Data is sent with fetch
pyngrok	Optional public link for demos (run_public.py)
Git and GitHub	Version control and hosting of the source code

5. System Architecture

The app follows a simple client-server design. The browser shows the page and talks to the Flask server through JSON requests. The server is split into small modules, each with one job. Only **gemini_service.py** talks to the AI and only **database.py** touches the database.

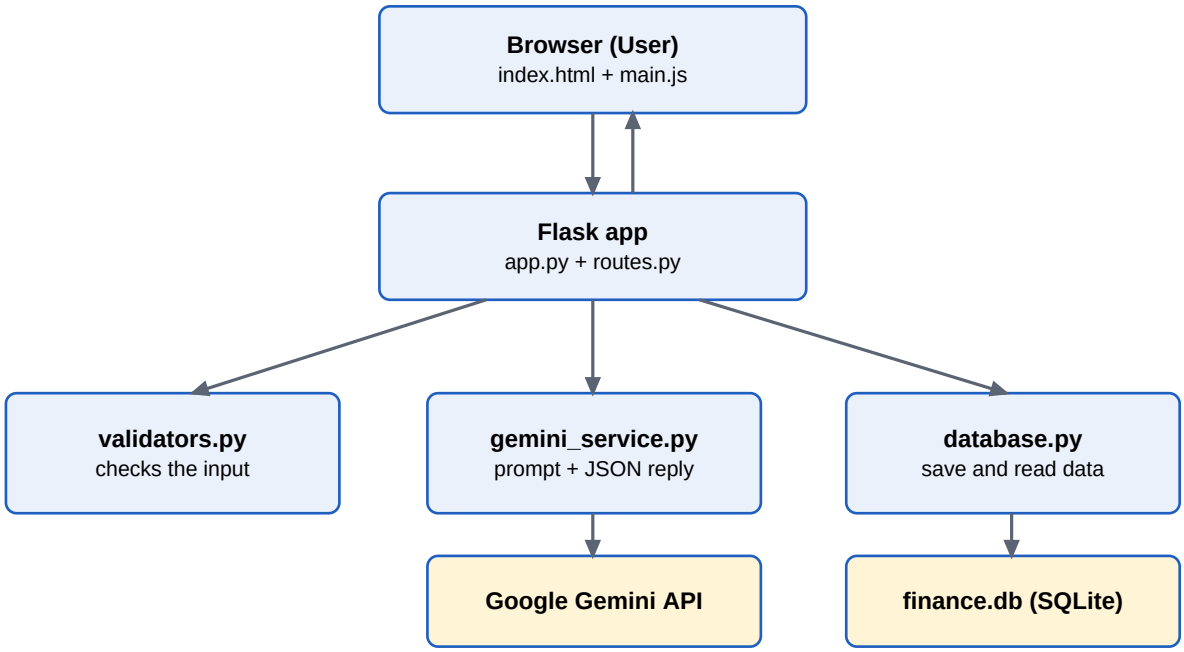


Figure: Architecture of the application

6. Project Structure

```
Personal-Finance-Advisor-Bot/
|-- app.py               Entry point, creates the Flask app
|-- config.py            Settings, guidelines, goal descriptions
|-- routes.py            Routes: /, /analyse, /history, /summary, /generate
|-- gemini_service.py    Gemini client, prompt builder, JSON extraction
|-- database.py          SQLite: create table, save, read
|-- validators.py        Checks the expense input
|-- test_app.py          21 automated tests
|-- run_public.py        Public link with ngrok
|-- requirements.txt     Python packages
|-- .env.example         Template for secret settings
|-- .gitignore           Files that are not uploaded to GitHub
|-- README.md            Setup and usage guide
|-- templates/
|   `-- index.html       The web page
|-- static/
|   |-- css/style.css    Styling
|   `-- js/main.js       Form, fetch calls, result display
`-- docs/
    |-- Project_Report.pdf
    `-- screenshots/
```

Why separate modules? A single large file is hard to read and test. With one file per responsibility, a change to the AI prompt touches only gemini_service.py, a change to storage touches only database.py, and so on.

7. Working of the System

- **Step 1 - Input.** The user enters income, expenses and picks a goal (emergency fund, vacation, gadget purchase or investment).
- **Step 2 - Request.** main.js sends the data to the ***/*analyse** route as JSON.
- **Step 3 - Validation.** The server checks that income is between 1 and 1,00,00,000 and that every expense is a valid number in range.
- **Step 4 - Prompt.** build_prompt() writes each category with its share of income and the guideline from config.py (for example rent: at most 30% of income) and adds the description of the chosen goal.
- **Step 5 - AI call.** ask_gemini() sends the prompt and asks for JSON only.
- **Step 6 - Parsing.** extract_json() reads the reply. If the model wraps the JSON in markdown fences or extra text, the JSON is still recovered.
- **Step 7 - Summary.** The app itself calculates total spent, money left over and the savings rate, so these figures do not depend on the AI.
- **Step 8 - Save.** The summary is stored in SQLite.
- **Step 9 - Display.** The page shows the suggested budget, an over/ok status for each category and 3 to 5 suggestions, then refreshes the monthly summary and recent analyses.

Spending guidelines used

Category	Guideline given to the AI
Rent	At most 30% of income
Food	Under 15% of income
Transport	Within 10% of income
Entertainment	Around 5-8% of income
Savings	At least 20% of income

8. Database Design

The app uses one SQLite table named **analyses**. The database file is created automatically the first time the app starts. Only the income, goal and the summary numbers are stored. The individual expense amounts are not saved.

Column	Type	Description
id	INTEGER (primary key)	Auto-increment number
created_at	TEXT	Date and time of the analysis
income	REAL	Monthly income entered by the user
goal	TEXT	Selected savings goal
summary	TEXT (JSON)	income, total_spent, left_over, savings_rate

The monthly summary groups saved rows by month and adds up income and spending to show net savings and the savings rate for the last 6 months.

9. Routes (API)

Route	Method	Purpose
/	GET	Shows the web page
/analyse	POST	Validates input, calls Gemini, returns budget, analysis, suggestions and summary, saves the analysis
/history	GET	Returns the last 5 analyses
/summary	GET	Returns monthly totals for the last 6 months
/generate	POST	Writes a short promotional post (under 100 words) for a finance product, as asked in the project specification

10. Input Validation and Error Handling

The app never trusts what the browser sends. Every request is checked before it reaches the AI.

Situation	What the app does
Income missing, zero, negative or too large	Returns a clear error message (HTTP 400)
Expense is not a number or is out of range	Returns an error naming the category (HTTP 400)
All expenses are empty	Asks the user to enter at least one expense
Category names contain symbols	Symbols are removed and the name is cut to 30 characters
GEMINI_API_KEY is missing	Returns a message telling the user to fix the .env file (HTTP 500)
Gemini cannot be reached	Returns a friendly error and logs the real one (HTTP 502)
AI reply is not valid JSON or has missing fields	Returns an error asking the user to try again (HTTP 502)
Database save fails	The error is logged but the user still gets the analysis

Security

- The API key lives only in the .env file, which is listed in .gitignore and is never uploaded.
- A .env.example file shows which settings are needed, with blank values.
- SQL queries use placeholders (?) instead of string joining.
- The local database file is also ignored by Git.

11. User Interface

The interface is a single responsive page. The top card holds the form. Below it are the monthly summary and the list of recent analyses. After an analysis, the suggested budget, the spending check and the suggestions appear on the same page.

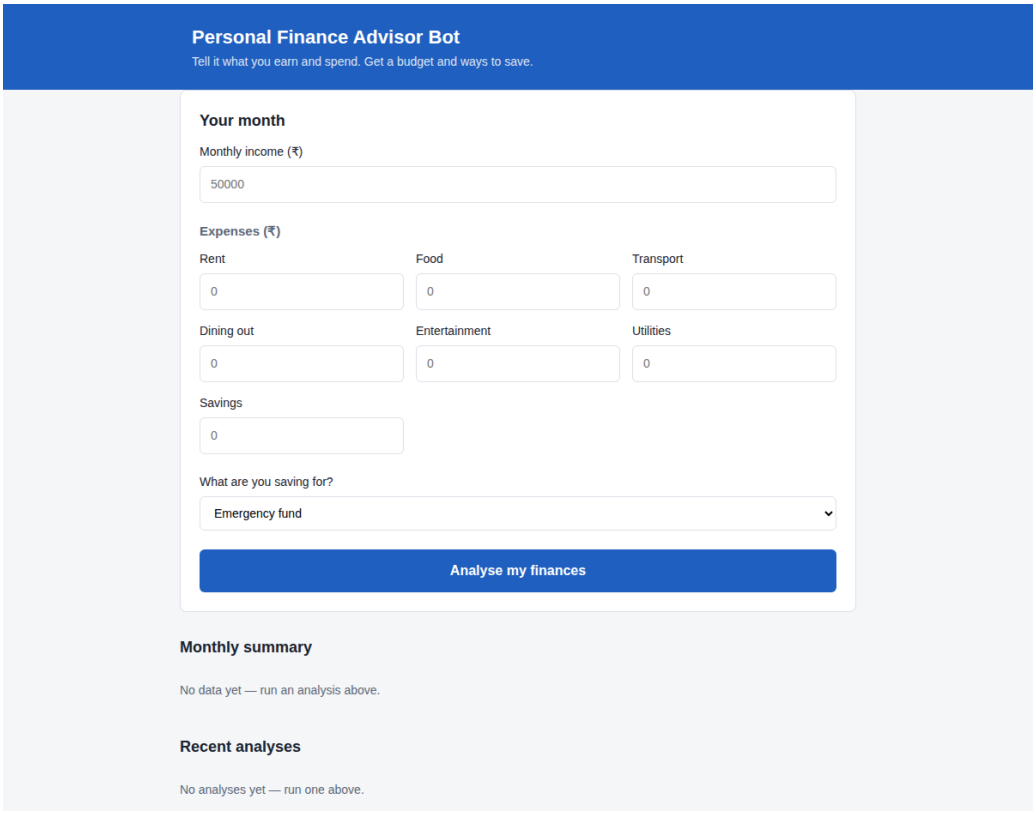


Figure 1: Input form of the application (first load, no data yet)

12. Testing

The project includes an automated test file, **test_app.py**, with 21 tests (run with *python -m unittest test_app.py -v*). The tests use Flask's test client. To make them repeatable, the Gemini call is replaced with a fixed reply and every test uses a temporary database. Real AI answers depend on the API key and model of the person running the app and will vary from run to run. The main cases covered are listed below.

Test	Expected result	Result
Open home page (/)	HTTP 200, page loads	Pass
Valid income and expenses to /analyse	HTTP 200 with budget, analysis, suggestions and summary	Pass
Income 0	HTTP 400	Pass
Expense value that is not a number	HTTP 400 with message naming the category	Pass
Summary values (50,000 income, 28,000 spent)	left_over = 22,000	Pass
/history after one analysis	One saved record returned	Pass

Test	Expected result	Result
/summary after one analysis	One month with income, expense and net savings	Pass
/generate with product text	HTTP 200 with post text	Pass
/generate with no product text	HTTP 400	Pass
AI reply wrapped in markdown fences	JSON still extracted	Pass
Prompt contains the income and guidelines	Prompt text is correct	Pass

13. Limitations and Future Scope

Limitations

- The advice comes from an AI model and is not professional financial advice.
- There is no login, so saved history is shared by everyone using the same running copy of the app.
- The app does not check that the AI's suggested budget adds up to the income.
- Expense categories are fixed on the form.

Future scope

- User accounts so every person sees only their own history.
- Server-side check that the suggested budget does not exceed income.
- Charts for spending trends over time.
- Custom categories and support for other currencies.
- Export of the report as PDF or Excel.
- Deployment on a cloud service with a permanent link.

14. Conclusion

The Personal Finance Advisor Bot turns a few numbers into a clear budget and practical next steps. It combines a generative AI model with exact calculations done in code, stores each analysis for later review, and handles bad input and AI failures without breaking. The code is split into small modules, which makes the project easy to understand and extend. The project shows how generative AI can support everyday financial decisions in a simple, usable way.

References

- Flask documentation - https://flask.palletsprojects.com
- Google Gemini API documentation - https://ai.google.dev
- SQLite documentation - https://www.sqlite.org/docs.html
- Project source code - https://github.com/alwaysonkar/Personal-Finance-Advisor-Bot